

WEEK 12 – PROJECT

(Learning CRUD Operations in MongoDB)

- By Sourav Kumar K

TASK 1 :

Create a new database & collections:

. New database created (studentdb)

```
mongosh mongodb://127.0.0.1:27017/ x + v
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\soura>mongosh
Current Mongosh Log ID: 6ac0aeld6cb880e6bf6df87d
Connecting to:   mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.12.0
Using MongoDB:  9.0.2
Using Mongosh:  2.12.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-10-03T12:10:08.578+05:30: Access control is not enabled for the database. Read and write access to data and configuration is
  unrestricted
  -----

test> use studentdb
switched to db studentdb
studentdb> |
```

. New collection created (students)

```
mongosh mongodb://127.0.0.1:27017/ x + v
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\soura>mongosh
Current Mongosh Log ID: 6ac0aeld6cb880e6bf6df87d
Connecting to:   mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.12.0
Using MongoDB:  9.0.2
Using Mongosh:  2.12.0

For mongosh info see: https://www.mongodb.com/docs/mongodb-shell/

-----
  The server generated these startup warnings when booting
  2026-10-03T12:10:08.578+05:30: Access control is not enabled for the database. Read and write access to data and configuration is
  unrestricted
  -----

test> use studentdb
switched to db studentdb
studentdb> db.createCollection("students")
{ ok: 1 }
studentdb> |
```

TASK 2:

Insert Operations:

. Insert one document into the student's collection.

```
mongosh mongodb://127.0.0.1:27017/ x + v
Microsoft Windows [Version 10.0.26200.9457]
(c) Microsoft Corporation. All rights reserved.

C:\Users\soura>mongosh
Current Mongosh Log ID: 6ac0aeld6cb880e6bf6df87d
Connecting to:  mongodb://127.0.0.1:27017/?directConnection=true&serverSelectionTimeoutMS=2000&appName=mongosh+2.12.0
Using MongoDB:  9.0.2
Using Mongosh:  2.12.0

For mongosh info see: https://www.mongodb.com/docs/mongosh-shell/

-----
The server generated these startup warnings when booting
2026-10-03T12:10:08.578+05:30: Access control is not enabled for the database. Read and write access to data and configuration is
unrestricted
-----

test> use studentdb
switched to db studentdb
studentdb> db.createCollection("students")
{ ok: 1 }
studentdb> db.students.insertOne({name:"Sourav",age:21,course:"Mern Stack",status:"enrolled"})
{
  acknowledged: true,
  insertedId: ObjectId('6ac0b0416cb880e6bf6df87e')
}
```

. Insert multiple documents with fields like name, age, course, status.

```
studentdb> db.students.insertMany([
  {name:"Praveen",age:20,course:"Python",status:"completed"},
  {name:"Prasath",age:21,course:"Mern Stack",status:"enrolled"},
  {name:"Dhilip",age:20,course:"python",status:"enrolled"},
  {name:"Mahi",age:21,course:"Mern Stack",status:"completed"}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac0b1ba6cb880e6bf6df87f'),
    '1': ObjectId('6ac0b1ba6cb880e6bf6df880'),
    '2': ObjectId('6ac0b1ba6cb880e6bf6df881'),
    '3': ObjectId('6ac0b1ba6cb880e6bf6df882')
  }
}
studentdb> |
```

TASK 3:

3. Read Operations:

- . Fetch all documents from the collection.

```
studentdb> db.students.find()
[
  {
    _id: ObjectId('6ac0b0416cb880e6bf6df87e'),
    name: 'Sourav',
    age: 21,
    course: 'Mern Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df87f'),
    name: 'Praveen',
    age: 20,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df880'),
    name: 'Prasath',
    age: 21,
    course: 'Mern Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df881'),
    name: 'Dhilip',
    age: 20,
    course: 'python',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df882'),
    name: 'Mahi',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  }
]
studentdb> |
```

- . Use filters to fetch specific data (e.g., students enrolled in “MERN Stack”).

```
studentdb> db.students.find({course: "Mern Stack"})
[
  {
    _id: ObjectId('6ac0b0416cb880e6bf6df87e'),
    name: 'Sourav',
    age: 21,
    course: 'Mern Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df880'),
    name: 'Prasath',
    age: 21,
    course: 'Mern Stack',
    status: 'enrolled'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df882'),
    name: 'Mahi',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  }
]
studentdb> |
```

TASK 4 :

4. Update Operations:

. Update a single document (e.g., change the status of a student to completed).

```
studentdb> db.students.updateOne({ name: "Sourav" }, { $set: { status: "completed" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
studentdb> |
```

. Update multiple documents at once.

```
studentdb> db.students.updateMany({ course: "Mern Stack" }, { $set: { status: "completed" } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 3,
  modifiedCount: 1,
  upsertedCount: 0
}
studentdb> db.students.find({course: "Mern Stack"})
[
  {
    _id: ObjectId('6ac0b0416cb880e6bf6df87e'),
    name: 'Sourav',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df880'),
    name: 'Prasath',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df882'),
    name: 'Mahi',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  }
]
studentdb> |
```

TASK 5:

5. Delete Operations:

. Delete one document based on a condition.

```
studentdb> db.students.deleteOne({ name: "Mahi" })
{ acknowledged: true, deletedCount: 1 }
studentdb> db.students.find({course: "Mern Stack"})
[
  {
    _id: ObjectId('6ac0b0416cb880e6bf6df87e'),
    name: 'Sourav',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df880'),
    name: 'Prasath',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  }
]
studentdb> |
```

. Delete all documents in the collection (for practice purpose only).

```
libraryDB> use studentdb
switched to db studentdb
studentdb> db.students.deleteMany({})
{ acknowledged: true, deletedCount: 4 }
studentdb> |
```

TASK 6 :

6. Query Operators:

. Practice using \$gt, \$lt, \$in, \$and, \$or, \$exists in different queries.

\$gt

```
studentdb> db.students.find({ age: { $gt: 20 } })
[
  {
    _id: ObjectId('6ac0b0416cb880e6bf6df87e'),
    name: 'Sourav',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df880'),
    name: 'Prasath',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  }
]
studentdb> |
```

\$lt

```
studentdb> db.students.find({ age: { $lt: 22 } })
[
  {
    _id: ObjectId('6ac0b0416cb880e6bf6df87e'),
    name: 'Sourav',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df87f'),
    name: 'Praveen',
    age: 20,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df880'),
    name: 'Prasath',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df881'),
    name: 'Dhilip',
    age: 20,
    course: 'python',
    status: 'enrolled'
  }
]
studentdb> |
```

\$in

```
studentdb> db.students.find({course: { $in: ["Mern Stack", "Python"] }})
[
  {
    _id: ObjectId('6ac0b0416cb880e6bf6df87e'),
    name: 'Sourav',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df87f'),
    name: 'Praveen',
    age: 20,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df880'),
    name: 'Prasath',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  }
]
studentdb> |
```

\$and

```
studentdb> db.students.find({$and: [{ age: { $gt: 20 } }, { status: "enrolled" } ]})
[
  {
    _id: ObjectId('6ac0bb3593a873b8696cbbb9'),
    name: 'Prasath',
    age: 21,
    course: 'Mern Stack',
    status: 'enrolled'
  }
]
studentdb> |
```

\$or

```
studentdb> db.students.find({$or: [{ course: "Mern Stack" }, { course: "Python" } ]})
[
  {
    _id: ObjectId('6ac0b0416cb880e6bf6df87e'),
    name: 'Sourav',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df87f'),
    name: 'Praveen',
    age: 20,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df880'),
    name: 'Prasath',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  }
]
studentdb> |
```

\$exists

```
studentdb> db.students.find({age: { $exists: true } })
[
  {
    _id: ObjectId('6ac0b0416cb880e6bf6df87e'),
    name: 'Sourav',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df87f'),
    name: 'Praveen',
    age: 20,
    course: 'Python',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df880'),
    name: 'Prasath',
    age: 21,
    course: 'Mern Stack',
    status: 'completed'
  },
  {
    _id: ObjectId('6ac0b1ba6cb880e6bf6df881'),
    name: 'Dhilip',
    age: 20,
    course: 'python',
    status: 'enrolled'
  }
]
studentdb> |
```

TASK 7:

7. Use Case:

. Create a small use case like a Library System or E-Commerce product listing and apply insert, update, and search operations.

```
mongosh mongod://127.0.0.1 x + v
status: 'enrolled'
}
]
studentdb> use libraryDB
switched to db libraryDB
libraryDB> db.createCollection("books")
{ ok: 1 }
libraryDB> db.books.insertMany([
  {title: "The Inner Alchemist",author: "Paulo escobar",category: "Fiction",price: 350,available: true},
  {title: "Good Habits",author: "James bond",category: "Self Help",price: 500,available: true},
  {title: "Computer Coding",author: "Robert Martin",category: "Programming",price: 650,available: false},
  {title: "JavaScript Guide",author: "David Roacher",category: "Programming",price: 700,available: true}
])
{
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ac0b87d6cb880e6bf6df883'),
    '1': ObjectId('6ac0b87d6cb880e6bf6df884'),
    '2': ObjectId('6ac0b87d6cb880e6bf6df885'),
    '3': ObjectId('6ac0b87d6cb880e6bf6df886')
  }
}
libraryDB> db.books.find({ category: "Programming" })
[
  {
    _id: ObjectId('6ac0b87d6cb880e6bf6df885'),
    title: 'Computer Coding',
    author: 'Robert Martin',
    category: 'Programming',
    price: 650,
    available: false
  },
  {
    _id: ObjectId('6ac0b87d6cb880e6bf6df886'),
    title: 'JavaScript Guide',
    author: 'David Roacher',
    category: 'Programming',
    price: 700,
    available: true
  }
]
libraryDB> |
```

```
libraryDB> db.books.updateOne({ title: "Computer Coding" }, { $set: { available: true } })
{
  acknowledged: true,
  insertedId: null,
  matchedCount: 1,
  modifiedCount: 1,
  upsertedCount: 0
}
libraryDB> |
```

CRUD OPERATIONS COMPLETED